

AWS RAG Project

Import Libraries:

```
import json
import os
import boto3
import streamlit as st

from dotenv import load_dotenv
load_dotenv()
```

Embedding Models:

- Let's try with 2 models:
 - Bedrock embeddings
 - Gemini embeddings

```
pip install -U langchain-aws
```

```
from langchain_aws import BedrockEmbeddings
from langchain_google_genai import GoogleGenerativeAIEmbeddings
```

Import Bedrock:

```
from langchain_community.llms import Bedrock
```

Data ingestion libraries:

```
import numpy as np
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
from langchain_community.document_loaders import PyPDFDirectoryLoader
```

Vector embeddings:

```
from langchain_community.vectorstores import FAISS
```

LLMs:

```
from langchain_core.prompts import PromptTemplate
from langchain.chains import create_retrieval_chain
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda
```

create_retrieval_chain

- **Retrieves Information:** It allows you to query a **document store** or **vector store** to fetch relevant information based on a user query (retrieval step).
- **Generates a Response:** Once relevant documents are retrieved, it passes them to a language model (e.g., GPT) to generate a coherent response that uses this information (generation step).

```
chain = create_retrieval_chain(llm, vector_store)
```

Bedrock Client:

```
bedrock = boto3.client("bedrock-runtime")
```

Embeddings:

```
bedrock_embeddings=BedrockEmbeddings(model_id="amazon.titan-embed-t
ext-v1",client=bedrock)
google_embeddings = GoogleGenerativeAIEmbeddings(model="models/gemi
ni-embedding-001")
```



For Google embeddings, we need API key. Therefore, this won't work except loading the `load_dotenv()`

Data Ingestion:

- We'll write a function
- We have a folder called "data", from that folder, we'll pick the files

```
def data_ingestion():  
    loader = PyPDFDirectoryLoader("AWS_App/data")  
    documents = loader.load()  
  
    #split the data  
    text_splitter= RecursiveCharacterTextSplitter(chunk_size = 4000, chunk_overlap=200)  
    splitted_docs = text_splitter.split_documents(documents)  
    return splitted_docs
```

Vector embeddings & Vector store:

```
def get_vector_store(splitted_docs):  
    faiss_vectorstore = FAISS.from_documents(splitted_docs, embeddings)  
    #Save Locally  
    faiss_vectorstore.save_local("AWS_App/faiss_index")  
    return faiss_vectorstore
```

Define 2 models (Claude & Llama)

```
def mistral():  
    llm = Bedrock(model_id= "mistral.mistral-7b-instruct-v0:2",  
                  client = bedrock,  
                  model_kwargs= {"max_tokens": 250})
```

```
return llm
```

```
def llama():
```

```
    llm = Bedrock(model_id= "meta.llama3-8b-instruct-v1:0",  
                  client = bedrock,  
                  model_kwargs= {"max_gen_len": 250})  
    return llm
```

For Mistral → Use `max_tokens`

(https://docs.aws.amazon.com/bedrock/latest/userguide/bedrock-runtime_example_bedrock-runtime_InvokeModel_MistralAi_section.html)

```
# Set the model ID, e.g., Mistral Large.  
model_id = "mistral.mistral-large-2402-v1:0"  
  
# Define the prompt for the model.  
prompt = "Describe the purpose of a 'hello world' program in one line."  
  
# Embed the prompt in Mistral's instruction format.  
formatted_prompt = f"<s>[INST] {prompt} [/INST]"  
  
# Format the request payload using the model's native structure.  
native_request = {  
    "prompt": formatted_prompt,  
    "max_tokens": 512,  
    "temperature": 0.5,  
}  
  
# Convert the native request to JSON.  
request = json.dumps(native_request)
```

For Llama → Use `max_gen_len`

(https://docs.aws.amazon.com/bedrock/latest/userguide/bedrock-runtime_example_bedrock-runtime_InvokeModel_MetaLlama3_section.html)

```

# Format the request payload using the model's native structure.
native_request = {
    "prompt": formatted_prompt,
    "max_gen_len": 512,
    "temperature": 0.5,
}

# Convert the native request to JSON.
request = json.dumps(native_request)

try:
    # Invoke the model with the request.
    response = client.invoke_model(modelId=model_id, body=request)

```

Prompt Template & Prompt

```
prompt_template = """
```

Human: Based on the following context, provide a concise summary that directly
Your summary should be approximately 250 words and include key details from 1
If the answer cannot be found in the provided context, state that you do not know

```
<context>
```

```
{context}
```

```
</context>
```

```
Question: {question}
```

```
Assistant:
```

```
"""
```

Prompt:

```
prompt = PromptTemplate(template=prompt_template,
                        input_variables= ["context", "question"])
```

Response

```
def get_response(llm, retriever, query):
```

```
    chain = create_retrieval_chain(retriever, prompt | llm | StrOutputParser())
    response = chain.invoke({"question": query})
    return response["answer"]
```

`create_retrieval_chain` function expects two main arguments: the `retriever` as the first argument, and then the rest of the chain (prompt, LLM, output parser) as the second argument.

Create Streamlit App

```
def main():
```

```
    st.title("RAG Chatbot with AWS Bedrock")
```

```
    st.write("Ask a question about your documents:")
    user_question = st.text_area("Your Question:", height=70)
```

```
    if st.button("Vectorize"):
        with st.spinner("Generating answer..."):
            splitted_docs = data_ingestion()
            get_vector_store(splitted_docs)
            st.success("Vector Database Created")
```

```
    if st.button("Mistral Output"):
        with st.spinner("Processing..."):
            faiss_index = FAISS.load_local("AWS_App/faiss_index", embeddings, allow
            llm = mistral()
            st.success(get_response(llm, faiss_index, user_question))
```

```
    if st.button ("Llama Output"):
```

```

with st.spinner("Processing..."):
    faiss_index = FAISS.load_local("AWS_App/faiss_index", embeddings, allow_dangerous_deserialization=True)
    llm = Llama()
    st.success(get_response(llm, faiss_index, user_question))

if __name__ == "__main__":
    main()

```

!!!IMP:

- `allow_dangerous_deserialization=True` : The code will not load the pickle file if you don't write this
- `.as_retriever()` : Mandatory. Because LangChain expects a Retriever wrapper (like `FAISS.as_retriever()`)

Entire Code:

```

import json
import os
import boto3
import streamlit as st

from dotenv import load_dotenv
load_dotenv()

#Embedding
from langchain_aws import BedrockEmbeddings
from langchain_google_genai import GoogleGenerativeAIEmbeddings

```

```

from langchain_community.llms import Bedrock

import numpy as np
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFDirectoryLoader

from langchain_community.vectorstores import FAISS

#LLMs
from langchain_core.prompts import PromptTemplate
from langchain.chains import create_retrieval_chain
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda

#Bedrock Client

st.set_page_config(page_title="RAG Chatbot with AWS Bedrock", layout="wide")

bedrock = boto3.client("bedrock-runtime")

# Embeddings
bedrock_embeddings=BedrockEmbeddings(model_id="amazon.titan-embed-text-v1")
google_embeddings = GoogleGenerativeAIEmbeddings(model="models/gemini-1.5-pro-latest")

embedding_options = ["Bedrock Embeddings", "Google Embeddings"]

selected_embedding = st.selectbox("Please select the embedding model", embedding_options)

if selected_embedding == "Bedrock Embeddings":
    embeddings = bedrock_embeddings
else:
    embeddings = google_embeddings

# Data Ingestion

```

```

def data_ingestion():
    loader = PyPDFDirectoryLoader("AWS_App/data")
    documents = loader.load()

    #split the data
    text_splitter = RecursiveCharacterTextSplitter(chunk_size = 4000, chunk_overlap = 200)
    splitted_docs = text_splitter.split_documents(documents)
    return splitted_docs

# Vector embeddings & Vector store

def get_vector_store(splitted_docs):
    faiss_vectorstore = FAISS.from_documents(splitted_docs, embeddings)
    #Save Locally
    faiss_vectorstore.save_local("AWS_App/faiss_index")
    return faiss_vectorstore

#LLMs

def mistral():
    llm = Bedrock(model_id= "mistral.mistral-7b-instruct-v0:2",
                  client = bedrock,
                  model_kwargs= {"max_tokens": 250})
    return llm

def llama():
    llm= Bedrock(model_id= "meta.llama3-8b-instruct-v1:0",
                  client = bedrock,
                  model_kwargs= {"max_gen_len": 250})
    return llm

# Prompt Template
prompt_template = """
Human: Based on the following context, provide a concise summary that directly answers the question.
Your summary should be approximately 250 words and include key details from the context.
If the answer cannot be found in the provided context, state that you do not know.
"""

```

```
<context>
{context}
</context>
```

```
Question: {question}
```

```
Assistant:
```

```
"""
```

```
prompt = PromptTemplate(template=prompt_template,
                          input_variables= ["context", "question"])
```

```
#Response
```

```
def get_response(llm, retriever, query):
```

```
    # Wrap retriever in a RunnableLambda so that it can become part of a chain
```

```
    retriever_runnable = RunnableLambda(lambda x: retriever.get_relevant_documents(x))
```

```
    # Create the chain using the correct format
```

```
    chain = create_retrieval_chain(retriever_runnable, prompt | llm | StrOutputParser)
```

```
    # Now invoke the chain
```

```
    response = chain.invoke({"question": query})
```

```
    # return response["answer"] # assuming your final output has a key 'answer'
```

```
    return response['answer']
```

```
# Create Streamlit App
```

```
def main():
```

```
    st.title("RAG Chatbot with AWS Bedrock")
```

```
    st.write("Ask a question about your documents:")
```

```
    user_question = st.text_area("Your Question:", height=70)
```

```
    if st.button("Vectorize"):
```

```
        with st.spinner("Creating Vector Database..."):
```

```

splitted_docs = data_ingestion()
get_vector_store(splitted_docs)
st.success("Vector Database Created")

if st.button("Mistral Output"):
    with st.spinner("Processing..."):
        faiss_index = FAISS.load_local("AWS_App/faiss_index", embeddings, allow_dangerous_deserialization=True)
        llm = mistral()
        st.success(get_response(llm, faiss_index, user_question))

if st.button("Llama Output"):
    with st.spinner("Processing..."):
        faiss_index = FAISS.load_local("AWS_App/faiss_index", embeddings, allow_dangerous_deserialization=True)
        llm = llama()
        st.success(get_response(llm, faiss_index, user_question))

if __name__ == "__main__":
    main()

```

Code Explanation:

```
faiss_index = FAISS.load_local("AWS_App/faiss_index", embeddings, allow_dangerous_deserialization=True)
```

◆ **load_local("AWS_App/faiss_index", embeddings, allow_dangerous_deserialization=True)**

This means:

- You already created a **vector database** and saved it locally at:
`AWS_App/faiss_index`
- `load_local(...)` is loading this index from your local disk.

- ◆ Internally, that "faiss_index" is a collection of **vectors**.

`.as_retriever()` :

- Now you've loaded your super-fast FAISS index. But the index itself just knows how to find similar numerical vectors.
- `as_retriever()` is like adding a "librarian" interface to your FAISS index. It wraps the FAISS index in a way that LangChain understands as a **"retriever."**
- A **retriever** is a LangChain component whose *only job* is to take a question (your "query") and go "retrieve" the most relevant chunks of text (your "context") from your knowledge base (your FAISS index).
- So, this whole line takes your saved numerical text summaries, loads them, and prepares them to act as a "smart search engine" that can find relevant documents for any given question.

```
retriever_runnable = RunnableLambda(lambda x: retriever.get_relevant_documents(x["question"]))
```

- `x` is a dictionary like this: `{"question": "What is AI?"}`
- This function says:
 - Take `x["question"]` (i.e., the question),
 - **There could be multiple variables in the dictionary. Our query is in the `question` variable.**
 - Pass it to `retriever.get_relevant_documents(...)`
 - **That will return a list of chunks from your FAISS vector database that are similar to the question.**
 - **What it does?:** This line essentially says: "Create a step in my AI process that, when given an input dictionary containing a 'question' key, will use my `retriever` to find and return relevant documents based on that 'question'."

RunnableLambda(...)

- This wraps the function in a **LangChain-compatible format**.
- Now you can use it inside a **chain**.



retriever_runnable gets the **question**, uses your **faiss_index**, and returns relevant **context** documents.

Basics of

`create_retrieval_chain`

`create_retrieval_chain` *simplifies* building a **RAG**

It's a **helper function** in LangChain to:

1. Get relevant documents using a **retriever**
2. Inject those documents into a **prompt** (usually with `{context}` placeholder)
3. Run the prompt through a **LLM**
4. **Parse** the output (clean/format it)

◆ How does it work?

python

Copy Edit

```
create_retrieval_chain(
    retriever,                # your retriever_runnable
    prompt | llm | StrOutputParser() # how you want to handle the question + documents
)
```

➤ **`prompt | llm | StrOutputParser()` is a chain of operations:**

- `prompt`: A PromptTemplate that includes `{context}` and `{question}`
- `| llm`: Passes the filled prompt to the language model
- `| StrOutputParser()`: Converts LLM response into plain string

When and where does `{context}` get filled?

✓ It happens automatically:

- LangChain **auto-fills** `{context}` by calling the retriever:

```
retriever.get_relevant_documents(question)
```

- Then it **injects** the text of those documents into `{context}` placeholder in your prompt.

Overall Flow of `chain = create_retrieval_chain(...)` :

- You invoke the `chain` with a `question`.
- The `create_retrieval_chain` automatically understands that `retriever_runnable` needs to run first with the `question`.
- `retriever_runnable` gets the `question`, uses your `faiss_index`, and returns relevant `context` documents.
- `create_retrieval_chain` then automatically takes the original `question` and the newly retrieved `context` and feeds them into your `prompt` template.
- The `prompt` generates a full message for the LLM.
- This message goes to the `llm`, which generates the answer.
- The `StrOutputParser` ensures the answer is a simple string.
- Finally, this string is returned as the `answer` when you `invoke` the chain.

Best Combination:

- Google + Llama

Best: Google (Llama)

- Most accurate, complete, and well-explained.
- Captures **Nagel's unique position** (our inability to understand bat-consciousness), his proposal of **objective phenomenology**, and its importance for **philosophy of mind**.

Second Best: Google (Mistral)

- Concise and accurate, but slightly less comprehensive.

Third: Bedrock (Llama)

- Decent for general understanding, but lacks detail and nuance.

Least Relevant: Bedrock (Mistral)

- Goes off-track by focusing on physicalism, which is the **opposite** of Nagel's argument.